EnrouteLogTech SaaS Architecture v1

Core Philosophy

Keep everything simple.

Avoid:

- Microservices
- Kubernetes
- Multiple databases
- Multiple backends

Use:

- One VPS
- One Frontend
- One Backend
- One Database

Build fast. Ship fast.

Tech Stack

Frontend

- Next.js
- TypeScript
- Tailwind CSS
- Shadcn UI
- TanStack Query
- React Hook Form
- Zod

Backend

- Express.js
- TypeScript
- JWT Authentication

Database

- MongoDB

Storage

- Cloudflare R2

Email

- Resend

Infrastructure

- Docker
- Nginx
- GitHub Actions

System Architecture

Users

↓

Cloudflare DNS

↓

Nginx

↓

Next.js Frontend

↓

Express API

↓

MongoDB

Files

↓

Cloudflare R2

Code

↓

GitHub

Repository Structure

enroutelogtech/

frontend/

backend/

infrastructure/

docker-compose.yml

.env

.github/

README.md

Frontend Structure

frontend/

src/

app/

(auth)

customer/

transporter/

driver/

admin/

components/

layouts/

hooks/

services/

types/

utils/

Routing Strategy

Customer

/customer

Transporter

/transporter

Driver

/driver

Admin

/admin

Never use large-scale conditional rendering.

Each role gets dedicated pages.

Backend Structure

backend/

src/

modules/

auth/

users/

companies/

rfqs/

bids/

contracts/

indents/

trips/

vehicles/

drivers/

documents/

notifications/

reports/

admin/

Module Structure

rfqs/

rfq.controller.ts

rfq.service.ts

rfq.model.ts

rfq.routes.ts

rfq.validation.ts

Repeat for every module.

User Architecture

Single User Collection

users

Fields

- Name
- Email
- Password
- CompanyId
- Role
- Status

Roles

ADMIN

CUSTOMER

TRANSPORTER

DRIVER

FLEET_OWNER

Company Architecture

companies

Fields

- Company Name
- GST
- PAN
- Company Type
- Status

Types

CUSTOMER

TRANSPORTER

BROKER

Authentication Flow

Register

↓

Email Verification

↓

Pending Approval

↓

Admin Approval

↓

Login

↓

JWT Token

↓

Role Redirect

File Storage

Cloudflare R2

Store

- POD
- LR
- Invoice

- RC
- Insurance
- Driver License
- Contract PDF
- Excel Uploads

MongoDB Stores

- File Name
- File URL
- Uploaded By
- Uploaded At

Never store files inside MongoDB.

Docker Architecture

Services

frontend

backend

mongodb

nginx

All services run through Docker Compose.

Environment Strategy

Local Development

Docker MongoDB

Production

Docker MongoDB

Only .env values change.

Never maintain separate database architectures.

CI/CD Flow

Local Development

↓

Git Commit

↓

GitHub Push

↓

GitHub Actions

↓

SSH To VPS

↓

Git Pull

↓

Docker Compose Build

↓

Deploy

Backup Strategy

Code

GitHub

Database

Daily mongodump

Create ZIP

Upload To R2

Keep Last 7 Backups

Files

Stored In R2

No VPS dependency

Development Order

Phase 1

Infrastructure

- VPS
- Docker
- Nginx
- Domain
- SSL
- CI/CD

Phase 2

Authentication

- Register
- Login
- Email Verification
- Admin Approval

Phase 3

Company System

- Companies
- Team Users
- Roles

Phase 4

Contract RFQ

- Create RFQ
- Excel Upload
- Validation
- Select Transporters
- Start Bidding

Phase 5

Transporter Portal

- RFQ List
- Place Bid
- Bid History
- Award Contract

Phase 6

Operations

- Indent
- Vehicle Assignment
- Driver Assignment
- POD Upload
- Trip Completion

First Milestone

Do NOT start business logic until these work:

✓ Docker Running

✓ MongoDB Running

✓ Next.js Running

✓ Express Running

✓ Nginx Running

✓ Domain Connected

✓ HTTPS Enabled

✓ GitHub Actions Working

✓ Cloudflare R2 Upload Working

Once these are complete:

Start Login System Development.

MVP Goal

Customer Creates RFQ

↓

Transporter Places Bid

↓

Customer Awards Contract

↓

Contract Created

Ship this before building advanced features.